PROGETTO DI ROBOTICA MEDICA

Robot a tendini con molla in serie

Guida unica: SOFA, glossario e nuova distribuzione del lavoro

Elaborato di riferimento
Progetto 1 - Sezione A: molla in serie passiva, tubo continuo

Gruppo
Anna - Josh - Annalisa - Annapia

Corso
Progetti Tecnici: Robotica Medica - A.A. 2026/2027
Docente: Fanny Ficuciello

Versione della guida
5 novembre 2026

Documento di lavoro per un gruppo che parte da zero con SOFA. Segue la logica a fasi del piano originale, ma corregge le parti che non permetterebbero di dimostrare i requisiti della traccia.

IMPORTANTE
Questa guida è un piano tecnico-operativo, non attesta che una scena SOFA sia già stata eseguita o validata. I nomi dei componenti e alcuni dettagli API devono essere verificati sulla stessa versione SOFA installata da tutto il gruppo.

Come usare questa guida

Il documento è organizzato per fasi, come il piano iniziale. Ogni fase indica: obiettivo, responsabile principale, procedura passo passo, file in ingresso, file in uscita e controllo da svolgere nella riunione. Le attività sono assegnate a una persona, ma i passaggi di integrazione sono sempre condivisi.

Indice operativo

Sezione	Contenuto
1	Requisiti della traccia e parametri congelati
2	Glossario operativo: simboli, unità e nomi nel codice
3	SOFA da zero: concetti, installazione, struttura cartelle
4	Ruoli, file e dipendenze tra i componenti
5	Fase 0 - ambiente e prima scena di prova
6	Fase 1 - geometria, FEM, tendine e moto libero
7	Fase 2 - SEA, Nitinol, controller, grafici e CSV
8	Fase 3 - ostacoli, contatto reale e campagna batch
9	Fase 4 - H, modello ridotto, validazione e intervalli ottimali
10	Riunioni, checklist, consegne e presentazione
11	Matrice finale di conformità alla traccia
12	Riferimenti tecnici ufficiali

Decisioni organizzative

- Un solo responsabile modifica ciascun file principale. Gli altri forniscono formule, dati o revisioni tramite interfacce concordate.
- La scena deve essere funzionante prima di lanciare decine di prove. Non si costruisce la campagna batch su dati simulati a mano.
- Ogni risultato inserito nella presentazione deve essere riconducibile a un CSV prodotto da una simulazione SOFA reale, con parametri e versione software registrati.
- Il gruppo usa un solo ambiente SOFA e una sola convenzione di unità e nomi delle colonne CSV.

Regola di avanzamento
Non si passa alla fase successiva perché il codice è stato scritto: si passa quando la checklist della fase è superata da tutti e il file di prova è salvato nella cartella condivisa.

1. Requisiti della traccia e parametri di riferimento

Il progetto assegnato è la Sezione A: molla in serie passiva con tubo continuo. Lo scopo è studiare il compromesso tra riduzione dell'isteresi nel moto libero e stabilità della forza durante il contatto con un oggetto rigido e uno soffice.

Parametri da mantenere come baseline

Grandezza	Valore della traccia	Nota pratica
Lunghezza totale	100 mm = 40 mm albero + 60 mm flessibili	L'asse del tubo è assunto lungo Z nel modello iniziale.
Diametro esterno / interno	1,93 mm / 1,49 mm	Spessore radiale = (1,93 - 1,49)/2 = 0,22 mm.
Materiale tubo	Nitinol: E carico 77 GPa; E scarico 65 GPa; ν=0,33	In unità mm-N-s: 77000 e 65000 MPa, ν=0,33.
Tendine	CableConstraint in forza; offset y_t=+0,6 mm; ancorato alla punta	Il percorso e l'ordine degli indici vanno verificati con una prova semplice.
Molla SEA	Lunghezza a riposo 20 mm; k baseline 1,2 N/mm; c baseline 1,5 N·s/mm	La molla è logica nel controller; non è necessario disegnarla in 3D.
Isteresi	Molla ±0,45 N; materiale ±0,40 N	Tenere separati i due contributi nel codice e nei parametri.
Attuazione	Passo tastiera 0,35 mm	Il valore di u_max e la velocità automatica vanno dichiarati come scelte di test, non come dati della traccia.
Oggetti	Dimensione caratteristica 20 mm; rigido e soffice	Per il soffice: E_s=5 kPa; ν=0,45.
Attrito	μ=0,2	Da impostare nel contact manager.
Contatto	10-50 mN; riferimento di mantenimento 20 mN	10 mN=0,010 N; 20 mN=0,020 N; 50 mN=0,050 N.
Stabilità	Oscillazione picco-picco <30% della media per 5 s, senza perdita di contatto	Va calcolata su una finestra reale di 5 s.
Target isteresi	H(k) ≤ 50% di H con attuazione diretta k=50 N/mm	La percentuale va calcolata rispetto al valore H misurato a k=50.

Conversioni e convenzione delle unità

Per la scena proposta si usa il sistema coerente mm-N-s. In tale sistema la tensione meccanica è espressa in N/mm², equivalente a MPa.

```
77 GPa = 77000 MPa = 77000 N/mm^2
65 GPa = 65000 MPa = 65000 N/mm^2
5 kPa  = 0.005 MPa = 0.005 N/mm^2
10 mN  = 0.010 N     20 mN = 0.020 N     50 mN = 0.050 N
```

Parametri non specificati: non inventarli senza documentarli

- Densità/massa: la traccia non assegna un valore. Il glossario riporta ρ Nitinol=6,45·10⁻⁶ kg/mm³ come riferimento, ma va confermato e convertito in modo coerente con le unità di SOFA prima di usarlo; per l'ostacolo soffice la densità resta da definire. Non inserire masse arbitrarie senza documentare l'ipotesi.
- u_max, velocità del ciclo automatico, durata del ciclo libero, orientamento della scena e condizione di appoggio dell'ostacolo soffice: se si usano 6 mm, 1,2 mm/s o altre scelte del piano iniziale, indicarle come parametri operativi del gruppo.

- Diametro e proprietà del tendine, precarico iniziale della molla e gestione del gioco: non sono completamente definiti nella traccia. Non nasconderli dentro il codice; registrarli in config.py e descriverli nella presentazione.

2. Glossario operativo: simboli, unità e nomi nel codice

Questa è la notazione comune per relazione, script, grafici e CSV. Per i valori numerici prevale la traccia specifica del Progetto 1 rispetto ai benchmark generali del glossario.

Simboli, significato e corrispondenza con il codice

Simboli	Significato / unità	Nome nel codice e riferimento del Progetto 1
OD, ID, t_wall	Diametri esterno/interno e spessore parete [mm]	OD, ID, t_wall = 1,93; 1,49; 0,22 mm.
L_tot, L_rigid, L_flex, y_t	Lunghezze e offset tendine [mm]	L_TOTAL=100; L_RIGID=40; L_FLEX=60; cable_offset_y=+0,60.
E_load, E_unload, ΔE, ν	Moduli di Young [MPa], differenza [MPa], Poisson [-]	E_LOAD=77000; E_UNLOAD=65000; ΔE=12000 MPa; ν tubo=0,33; ν soffice=0,45.
ρ	Densità volumetrica [kg/mm³]	Glossario: $6{,}45\cdot10^{-6}$ kg/mm³. Non inserirla in SOFA senza conversione coerente con mm-N-s.
u_2 (u), u_1, ΔL_s, u_dot	Comando motore, spostamento lato tendine, allungamento molla [mm], velocità [mm/s]	self.u2 (o self.u), self.u1, self.delta_Ls, self.u_dot. $\Delta L_s=\max(0,u_2-u_1)$; u_1 va riferito allo zero iniziale e col segno verificato.
T, T_max, dir	Trazione tendine [N], limite [N], direzione [-]	self.T; T≥0. dir=+1 carico e −1 scarico. T_max=10 N è il benchmark di sicurezza del glossario, da confermare come limite del progetto.
X, Y, Z, ΔY, ΔY_max	Coordinate punta e deflessione laterale [mm]	tip_x, tip_y, tip_z, delta_Y, delta_y_max. ΔY=Y_punta−Y_iniziale; leggere la punta dal tubo FEM/marker mappato.
k, L_0, c	Rigidezza [N/mm], lunghezza riposo [mm], smorzamento [N·s/mm]	k_spring=1,2 baseline; spring_rest_length=20; damping_c=1,5.
F_h-molla, F_h-mat, F_hyst,total	Isteresi molla, materiale e totale [N]	f_h_molla=±0,45; f_h_mat=±0,40; f_hyst_total=dir·(F_h_molla+F_h_mat).
E_s, μ, F_contatto	Modulo soffice [kPa], attrito [-], forza normale [mN]	E_s=5 kPa; mu=0,20; contact_force_mN target 10-50, mantenimento 20.
H, dt	Indice isteresi [%], passo temporale [s]	hysteresis_H_pct; dt=0,01 s come scelta operativa da verificare.

Formule e convenzioni

```
delta_Ls = max(0.0, u2 - u1)
F_hyst_total = dir * (F_h_molla + F_h_mat)
H = max_u |DeltaY_load(u) - DeltaY_unload(u)| / DeltaY_max * 100
Oscillazione_contatto = (F_max - F_min) / F_media * 100
```

Il contributo viscoso è dissipativo e deve opporsi alla velocità relativa della molla: il segno nel calcolo della tensione richiesta dipende dalla convenzione scelta per forza e spostamento. Definirla nel README e verificarla con prove di carico e scarico. La stabilità richiede oscillazione <30%, F_min>0 e contatto sempre attivo nella finestra di 5 s.

Valori del glossario generale diversi dalla traccia

Per il Progetto 1 usare sempre i valori specifici della traccia: ID=1,49 mm (non 1,50); t_wall=0,22 mm (non 0,215); E_unload=65 GPa (non 66); ν Nitinol=0,33 (non 0,30); k baseline=1,20 N/mm (non il benchmark generale 1,13). Il valore $\rho=6{,}45\cdot10^{-6}$ kg/mm³ è solo un riferimento del glossario: la traccia non specifica la densità e occorre verificarne la conversione prima di inserirla nel modello.

Non servono in questa Sezione A i parametri di tubo intagliato (d, s, h, N), TPU, tre tendini, VSA k(t), modello di Dahl (z, σ_0), Capstan (η_capstan) o diametro del lume: sono voci del glossario generale per altre configurazioni.

> Regola pratica
> La traccia specifica prevale sui benchmark del glossario. Dal glossario prendiamo soprattutto simboli, significati, unità e nomi coerenti per il codice.

3. SOFA da zero: concetti e ambiente

Che cos'è una scena SOFA

SOFA organizza una simulazione come un albero di nodi, chiamato scene graph. Il nodo root contiene il sistema; i nodi figli contengono il tubo, la visualizzazione, la collisione, il tendine e gli ostacoli. Ogni nodo può contenere componenti: mesh loader, MechanicalObject, solutori, force field, vincoli e mapping.

Concetto	Significato nel progetto
Node / rootNode	Contenitore della scena e dei suoi componenti.
MechanicalObject	Coordinate e gradi di libertà dei punti del tubo o dell'oggetto.
Topology	Collegamenti tra punti: tetraedri nel volume, triangoli sulla superficie.
FEMForceField	Calcola la risposta elastica del tubo o dell'ostacolo soffice.
FixedConstraint	Blocca i punti selezionati, per esempio i primi 40 mm.
CableConstraint	Modella il tendine e impone displacement oppure force.
Mapping	Collega rappresentazioni diverse: fisica, superficie visuale e superficie di collisione.
Collision pipeline	Individua le intersezioni e crea i vincoli di contatto.
Controller Python	Legge la tastiera, aggiorna il comando SEA e registra i dati.

Come si installa e si rende riproducibile l'ambiente

1. Scegliere una release SOFA e mantenerla identica sui quattro computer. Non iniziare con versioni diverse.
2. Verificare che siano disponibili almeno i plugin necessari per FEM, mesh, mapping, collisioni, solutori, SofaPython3 e SoftRobots. La lista precisa dipende dalla versione e dai componenti effettivamente usati.
3. Aprire runSofa e controllare che SofaPython3 sia caricato. La documentazione ufficiale spiega come controllare i plugin e aprire uno script Python in runSofa [S1].
4. Verificare che Python usato da SOFA sia compatibile con SofaPython3 e con le librerie che userete (per esempio NumPy e Matplotlib). Non dare per scontato che l'interprete Python installato separatamente sia quello incorporato in SOFA.
5. Salvare in README.txt la versione di SOFA, i plugin, il sistema operativo, il percorso di runSofa e le librerie Python disponibili.
6. Creare una scena minima che contenga solo rootNode, un AnimationLoop e un Controller che stampa una riga. Avviarla prima di importare Gmsh o caricare il tubo.

> Suggerimento per principianti
> Non scrivete subito tutta la scena. Aggiungete un componente alla volta, lanciate runSofa e controllate che il componente risulti valido. Quando compare un errore, correggete quello prima di aggiungere altro.

Struttura consigliata della cartella condivisa

```
progetto_sofa/
  README.txt
  config.py
  1_generate_meshes.py
  2_scene_builder.py
  3_tendon_controller.py
  4_main.py
  5_live_plotter.py
  6_generate_obstacles.py
  7_run_campaign.py
  8_signal_analysis.py
  9_reduced_model.py
  9b_validate_model.py
  10_plot_main_results.py
  11a_plot_cycles.py
  11b_plot_sensitivity.py
  meshes/
    tube_volumetric.vtk
    tube_surface.stl
    obstacle_rigid.stl
    obstacle_soft.vtk
  dataset/
    raw/
    summary_campaign.csv
    sensitivity_campaign.csv
  figures/
  video/
  presentation/
```

Usare Git o una cartella condivisa con versioni numerate. Non sovrascrivere i CSV grezzi: ogni prova deve avere un nome univoco che contenga scenario e parametri. Il file config.py è la fonte unica dei parametri numerici: evitare di riscrivere k, E o μ in molti script diversi.

4. Nuova distribuzione del lavoro e responsabilità

La ripartizione sotto assegna un proprietario per ogni file principale. Le parti teoriche vengono condivise, ma le modifiche al codice passano dal proprietario del modulo. In questo modo non ci sono quattro persone che modificano contemporaneamente lo stesso controller.

Annalisa - geometria, materiale e modello matematico

- Responsabile di config.py per i parametri fisici comuni, dopo accordo del gruppo.
- Responsabile di 1_generate_meshes.py e 6_generate_obstacles.py (Gmsh).
- Definisce la commutazione del Nitinol tra E_load ed E_unload e ne prepara il test; Annapia integra la regola nel controller.
- Responsabile di 9_reduced_model.py: modello H(k), identificazione Kt, Fh_molla e Fh_mat con dati di identificazione reali.
- Responsabile di 11b_plot_sensitivity.py, usando i risultati dell'analisi CSV. Consegna figure e commento sull'effetto di Fh_molla e c.

Annapia - controller, SEA e analisi dei segnali

- Responsabile unico di 3_tendon_controller.py: tastiera, carico/scarico, reset, manuale/autonomo e contact_hold.
- Implementa ΔLs, velocità della molla, tensione T, saturazione unilaterale e gestione coerente dell'isteresi, dopo avere concordato il riferimento iniziale.
- Integra nel controller i parametri E_load/E_unload preparati da Annalisa; non modifica la formula del materiale senza confronto.

- Responsabile di 8_signal_analysis.py: calcolo di H, rapporto rispetto a H(50), stabilità su 5 s e file riepilogativi.
- Responsabile di 9b_validate_model.py: verifica indipendente sui k=0,9 e 3,4 N/mm.
- Responsabile di 11a_plot_cycles.py: confronto dei cicli ΔY(u) e serie temporali di contatto, in collaborazione con Josh per il formato CSV.

Anna - scena SOFA, contatto e risultati principali

- Responsabile di 2_scene_builder.py: FEM del tubo, albero fisso, cable node, mapping visuale e di collisione.
- Integra la collision pipeline e gli ostacoli rigido e soffice; verifica che il contatto sia effettivo.
- Coordina con Josh la lettura della forza reale di contatto dal solver e la sua associazione al corretto vincolo.
- Responsabile di 10_plot_main_results.py: grafici H(k), oscillazione di contatto e intervalli validi per rigido, soffice e comune.

Josh - integrazione, logger e campagna di simulazione

- Responsabile di 4_main.py: importa i moduli, carica plugin, costruisce la scena e istanzia controller, logger e grafici.
- Responsabile di 5_live_plotter.py: grafici real-time con aggiornamento non bloccante.
- Responsabile del modulo logger (csv_logger.py): schema delle colonne, scrittura dati, flush periodico e chiusura file.
- Responsabile di 7_run_campaign.py: esegue vere simulazioni SOFA batch, una configurazione alla volta, e verifica i CSV creati.
- Raccoglie log, versioni e risultati grezzi; non modifica le formule di H o del modello ridotto.

Equilibrio del carico: perché questa divisione

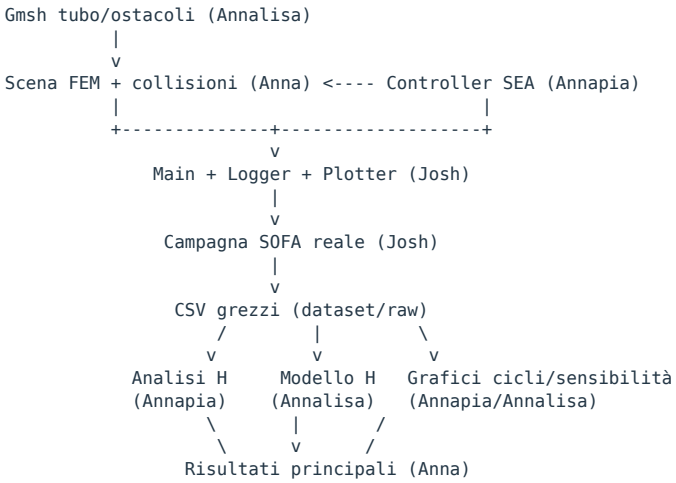
Membro	Area principale	Carico qualitativo	Motivo
Annalisa	Gmsh + Nitinol + modello H + sensibilità	Medio-alto	Due script geometrici, una parte costitutiva e la stima matematica.
Annapia	Controller completo + SEA + analisi + validazione + cicli	Alto	Un solo modulo centrale di controllo, più analisi dei dati con formule definite.
Anna	Scena FEM + collisioni + forza contatto + grafici target	Alto	È la parte più impegnativa di SOFA, ma è concentrata su scena e risultati fisici.
Josh	Main + logger + LivePlotter + batch reale	Alto	Infrastruttura e automazione, con interfacce definite dagli altri moduli.

Non è possibile garantire un numero identico di ore senza conoscere esperienza e velocità di ciascuno. Questa ripartizione è più equa perché distribuisce le attività difficili e assegna un unico responsabile ai file condivisi. Josh resta responsabile del batch, ma non anche dei grafici di sensibilità e dei cicli; Anna non deve scrivere il controller; Annalisa e Annapia si dividono modello e validazione.

5. Mappa dei file e dipendenze

File	Proprietario	Input necessari	Output / chi lo usa
config.py	Annalisa	Parametri fissati dal gruppo	Configurazione letta da tutti i moduli.
1_generate_meshes.py	Annalisa	Dimensioni tubo, Gmsh	tube_volumetric.vtk + tube_surface.stl → Anna.
6_generate_obstacles.py	Annalisa	Diametro e posizione oggetti	obstacle_rigid.stl + obstacle_soft.vtk → Anna.
2_scene_builder.py	Anna	Mesh tubo e ostacoli, config.py	Funzioni build_robot / add_obstacle → 4_main.py.
3_tendon_controller.py	Annapia	Nodi SOFA, parametri SEA/materiale	Controller → 4_main.py; scrive dati tramite logger.
csv_logger.py	Josh	Campi concordati con Annapia	CSV grezzi → Annapia, Annalisa, Anna.
5_live_plotter.py	Josh	Valori dal controller/logger	Grafici real-time → video e controllo simulazione.
4_main.py	Josh	Scene builder, controller, logger, plotter	Scena SOFA avviabile → prove manuali e batch.
7_run_campaign.py	Josh	4_main.py + config per prova	dataset/raw/*.csv → 8_signal_analysis.py.
8_signal_analysis.py	Annapia	CSV grezzi reali	summary_campaign.csv + sensitivity_campaign.csv.
9_reduced_model.py	Annalisa	H dei k di identificazione	Parametri Kt, Fh_molla, Fh_mat → validazione e grafici.
9b_validate_model.py	Annapia	Modello + k=0,9 e 3,4 misurati	Tabella errori percentuali <15%.
10_plot_main_results.py	Anna	Summary, modello H e sott modello contatto	H(k), stabilità, range kmin/kmax.
11a_plot_cycles.py	Annapia	CSV free/rigid/soft	Confronti ciclo e forza in 5 s.
11b_plot_sensitivity.py	Annalisa	sensitivity_campaign.csv	Grafici sensibilità Fh_molla e c.

Dipendenze fondamentali



Regola di integrazione
Prima di scrivere i moduli, concordare le firme delle funzioni e lo schema CSV. Nessuno deve cambiare nome a una colonna o a una variabile condivisa senza avvisare gli altri.

Schema minimo di CSV comune a tutti

```
time_s,scenario,k_N_mm,c_Ns_mm,Fh_spring_N,Fh_material_N,
direction,u_mm,delta_l_cable_mm,delta_Ls_mm,T_tendon_N,
tip_x_mm,tip_y_mm,tip_z_mm,delta_Y_mm,contact_normal_N,
contact_force_mN,contact_active,mode,run_id
```

Aggiungere solo colonne utili, ma mantenere sempre le stesse unità e gli stessi nomi in tutti i file. Se il contatto non è attivo, scrivere 0 e contact_active=False; non lasciare valori mancanti ambigui. Registrare anche parametri e identificativo della prova.

6. Fase 0 - Preparazione ambiente e primo test SOFA

Obiettivo della fase
Tutti e quattro devono aprire la stessa versione di SOFA, eseguire una scena Python minima e leggere la console senza errori. Non si parte ancora dalla mesh del robot.

Josh - coordinamento ambiente

1. Creare la cartella progetto e il README con versione SOFA, plugin, Python e percorsi.
2. Sul proprio computer aprire runSofa e lanciare l'opzione di aiuto da terminale per verificare che l'eseguibile funzioni. La documentazione ufficiale descrive -g batch e -n per eseguire un numero finito di step [S1].
3. Controllare con il Plugin Manager la presenza di SofaPython3 e SoftRobots; annotare i plugin mancanti.
4. Preparare 00_test_python.py con una scena minima e un controller che stampa 'step ok'.
5. Condividere istruzioni e screenshot della versione/installazione agli altri tre.

Tutti - prova individuale

1. Avviare runSofa e aprire la scena minima.
2. Premere Animate: la console deve stampare il messaggio del controller senza errori di import.
3. Verificare che il tasto di stop/reset della GUI funzioni e che la scena possa essere riaperta.
4. Scrivere nel README eventuali differenze tra computer; non proseguire finché non sono risolte.

Riunione 0 - 30/45 minuti, prima della Fase 1

- Confrontare versione SOFA, plugin e interprete Python sui quattro computer.
- Decidere cartella condivisa, nomi file, branch Git e convenzione mm-N-s.
- Confermare chi custodisce config.py e chi può modificarlo.
- Eseguire insieme 00_test_python.py su almeno due computer diversi; gli altri ripetono poi il test.

Uscita obbligatoria della Fase 0

README.txt compilato; ambiente replicabile; scena Python minima funzionante su tutti i PC. Se SofaPython3 o SoftRobots non sono caricati, non ha senso iniziare il controller o il CableConstraint.

7. Fase 1 - Geometria, FEM, tendine e moto libero

Obiettivo della fase
Visualizzare il tubo completo da 100 mm, fissare correttamente i primi 40 mm, deformare i restanti 60 mm con il tendine e comandarlo da tastiera. In questa fase si valida il modello di base senza ostacoli e senza SEA definitiva.

7.1 Annalisa - creare la mesh del tubo

File: 1_generate_meshes.py. Strumento: Gmsh/Python. Input: parametri geometrici della traccia.

1. Definire lunghezza 100 mm, raggio esterno 0,965 mm e raggio interno 0,745 mm.
2. Creare due cilindri coassiali lungo Z, entrambi lunghi 100 mm.
3. Sottrarre il cilindro interno da quello esterno con l'operazione booleana di Gmsh; sincronizzare il modello CAD.
4. Impostare la dimensione della mesh iniziale e generare una mesh 3D. Assicurarsi che la mesh volumetrica contenga tetraedri utilizzabili dal FEM.
5. Esportare tube_volumetric.vtk per il volume e tube_surface.stl per la superficie. Se la versione/loader non interpreta la topologia VTK come previsto, salvare anche il formato nativo Gmsh .msh e concordare con Anna il loader più robusto.
6. Aprire i file in ParaView o nel visualizzatore disponibile: verificare lunghezza, foro, spessore e continuità. Controllare che il file volumetrico non sia soltanto una superficie triangolare.
7. Fare almeno una verifica di convergenza: confrontare una simulazione con mesh iniziale e una più fine, mantenendo identici parametri; riportare la differenza dello spostamento di punta.

Output: tube_volumetric.vtk, tube_surface.stl, eventuale .msh, immagine di controllo e nota sulle dimensioni della mesh. Passaggio a Anna prima dell'integrazione.

7.2 Anna - costruire il tubo FEM in SOFA

File: 2_scene_builder.py. Non iniziare da tutta la scena: costruire il tubo a piccoli passaggi.

1. Creare il nodo figlio Tube e aggiungere un integratore implicito (EulerImplicitSolver) e un solutore lineare compatibile con la mesh e la versione installata (per esempio SparseLDLSolver).
2. Caricare tube_volumetric.vtk con MeshVTKLoader. Aggiungere la topologia tetraedrica coerente con il file e un MechanicalObject template Vec3.
3. Aggiungere TetrahedronFEMForceField con E di carico 77000 MPa, ν=0,33 e il metodo supportato (per esempio large). Verificare che il componente risulti Valid.
4. Aggiungere una massa coerente. La traccia non fornisce la densità: non usare 0,01 kg senza motivazione. Definire e documentare l'ipotesi oppure chiedere al docente il dato mancante.
5. Aggiungere BoxROI che selezioni i punti con Z tra 0 e 40 mm e FixedConstraint su quegli indici. Stampare il numero di nodi selezionati e visualizzare la ROI.
6. Aggiungere il nodo Visu con STL, OglModel e BarycentricMapping. Specificare esplicitamente input (FEM del Tube) e output (modello visuale) e verificare che la superficie segua il tubo.
7. Per il cavo creare MechanicalObject con punti interni spostati di +0,6 mm in Y. Aggiungere CableConstraint e mapping verso il tubo. Controllare l'ordine degli indici: la documentazione SoftRobots descrive gli indici dal capo del cavo al punto attuato; pullPoint e indice di attuazione devono essere coerenti [S4].
8. Per il test base iniziare con valueType='displacement' e un piccolo comando. Solo dopo avere verificato il verso del movimento si passerà a valueType='force' per la SEA.

Output: funzione build_robot(rootNode, path, config) che restituisce riferimenti al tubo, al tendine e al punto di punta. Non usare il punto finale del cavo come sostituto della punta del tubo.

7.3 Annapia - controller manuale iniziale

1. Creare una classe che eredita da Sofa.Core.Controller e inizializza u=0 e passo 0,35 mm. SofaPython3 supporta callback come onAnimateBeginEvent e gestori di eventi [S7].
2. Gestire freccia su/W per aumentare u, freccia giù/S per diminuire u senza andare sotto zero, R per azzerare.
3. Nel callback di animazione aggiornare il valore del CableConstraint in displacement mode.
4. Dopo il comando stampare u e le coordinate reali della punta del tubo FEM (marker o indice verificato), con unità mm.
5. Provare un solo step di comando alla volta e controllare che il moto corrisponda al verso atteso; se il tendine si allunga invece di accorciarsi, correggere convenzione e segno prima di proseguire.

7.4 Josh - main e integrazione iniziale

1. Creare 4_main.py e importare i moduli con percorsi relativi robusti.
2. Caricare i plugin necessari per Python, SoftRobots, FEM, mapping, mesh, visualizzazione e solutori.
3. Creare rootNode, impostare dt e gravità secondo il modello, aggiungere AnimationLoop e ConstraintSolver compatibili con i vincoli usati.
4. Chiamare build_robot() e istanziare il controller di Annapia. Chiamare le funzioni effettivamente: non basta definirle in un altro modulo.
5. Aprire la scena con runSofa, premere Animate e controllare console e scena grafica.

Riunione 1 - fine Fase 1

Chi	Porta alla riunione	Prova da fare insieme
Annalisa	Mesh volume + STL + report mesh	Caricare la mesh e controllare dimensioni e topologia.
Anna	Scena Tube + FEM + BoxROI + mapping	Verificare che i primi 40 mm siano fissi e la parte flessibile deformabile.
Annapia	Controller manuale	Premere W/S/R e verificare passo 0,35 mm e coordinate della punta.
Josh	4_main.py + README versione	Aprire la scena completa senza errori e salvare screenshot/video breve.

Checklist per passare alla Fase 2
Scena caricata senza errori; mesh volumetrica valida; ROI con nodi selezionati; tubo ancorato sui primi 40 mm; tendine che tira nel verso corretto; comandi tastiera funzionanti; coordinate della punta riferite al tubo, non al cavo.

8. Fase 2 - SEA, Nitinol, controller, grafici e CSV

Obiettivo della fase
Sostituire il controllo cinematico di prova con il controllo in forza della molla in serie, inserire la commutazione del Nitinol, completare il ciclo autonomo e produrre CSV e grafici in tempo reale.

8.1 Annalisa - legge del materiale Nitinol

1. In config.py definire E_load=77000 MPa, E_unload=65000 MPa e Fh_mat=0,40 N.
2. Definire una regola univoca per il ramo: delta_u positivo significa carico, delta_u negativo scarico; a delta_u circa zero mantenere l'ultimo ramo non nullo.
3. Scrivere una piccola funzione o una specifica da consegnare ad Annapia: in carico imposta E_load, in scarico E_unload, a riposo conserva il ramo precedente.
4. Chiarire nella relazione che questa è la rappresentazione semplificata prescritta dalla traccia (modulo diverso nei due rami + termine di isteresi), non un modello completo di superelasticità del Nitinol.
5. Verificare con Annapia che l'isteresi materiale non venga sommata due volte: una sola implementazione del contributo Fh_mat deve essere attiva.

8.2 Annapia - implementazione completa della molla SEA

La molla è modellata nel controller, non come mesh 3D. La grandezza fondamentale è l'allungamento effettivo tra il comando motore e l'accorciamento del tendine.

```
u2 = self.u
u1 = self.u1  # spostamento lato tendine, riferito allo zero iniziale
delta_Ls = max(0.0, u2 - u1)
v_spring = (delta_Ls - delta_Ls_prev) / dt
F_hyst_total = direction * (Fh_molla + Fh_mat)
# Il termine viscoso si oppone alla velocita relativa;
# applicare il segno secondo la convenzione definita nel README.
T_raw = k * delta_Ls + F_visc_equiv + F_hyst_total
T = min(T_max, max(0.0, T_raw))
```

1. Passare CableConstraint a valueType='force'. Non farlo finché la prova di displacement della Fase 1 non è superata.
2. Leggere il displacement restituito da CableConstraint e ricavare u1 come spostamento lato tendine rispetto al riferimento iniziale. Verificare sperimentalmente il segno con un piccolo comando; se l'output SOFA usa il segno opposto, convertirlo esplicitamente e documentarlo.
3. Calcolare ΔL_s=max(0,u2-u1) e la velocità usando dt reale dell'evento. Proteggere il calcolo da dt nullo; ΔL_s non deve diventare negativa.
4. Applicare k, c, Fh_molla e Fh_mat dai parametri centralizzati. Il termine viscoso deve opporsi alla velocità relativa; saturare T tra 0 e T_max (10 N è il riferimento del glossario, da confermare come limite di sicurezza del progetto).
5. Gestire l'avvio/reset: stato isteretico iniziale neutro; nessuna tensione iniziale non motivata. Concordare col docente o documentare la convenzione sul precarico e sul ramo a riposo.
6. Aggiornare il valore del CableConstraint con T e tenere aggiornati u_prev e delta_Ls_prev dopo il calcolo.
7. A ogni comando stampare u [mm], T [N], ΔLs [mm] e posizione punta [mm], come richiesto dalla traccia.

8.3 Annapia - modalità manuale, ciclo libero e contact_hold

Modalità	Comportamento richiesto	Verifica
manual	W/S o frecce modificano u a passi di 0,35 mm; R resetta.	u cresce e decresce, direction aggiornata correttamente.
free_cycle	Onda triangolare automatica 0 → u_max → 0 per misurare H.	Una salita e una discesa complete; nessun overshoot e nessuna falsa direzione.
contact_hold	Avvicinamento, contatto, raggiungimento del target 20 mN e mantenimento per 5 s.	La finestra di 5 s parte dopo l'assestamento; forza 10-50 mN e contatto sempre attivo.

Per contact_hold, se il comando deve mantenere 20 mN, non basta un'etichetta di modalità: usare un punto operativo calibrato o un controllo proporzionale/integrale sulla forza di contatto reale misurata. Registrare quando inizia la finestra di valutazione dei 5 s.

8.4 Josh - logger CSV e LivePlotter

1. Creare csv_logger.py separato dal controller. Inizializzare intestazione con i campi e le unità concordate nella mappa dei file.
2. A ogni step acquisire tempo, scenario, k, c, isteresi, direction, u, ΔL_cavo, ΔLs, T, coordinate punta, ΔY, forza normale di contatto, forza in mN e flag contact_active.
3. Scrivere i campioni nel CSV. Evitare flush su disco a ogni singolo step se rallenta la simulazione: usare un flush periodico (ad esempio ogni 50-100 righe) e chiusura sicura alla fine. Garantire che il file sia comunque salvato anche in caso di errore.
4. Creare 5_live_plotter.py. Un pannello per X/Y/ΔY; un pannello per u/ΔLs/T; un pannello separato per F_contatto. Non mettere mm, N e mN sullo stesso asse senza una scala secondaria chiaramente dichiarata.
5. Aggiornare il grafico ogni 15 step o con frame-skipping analogo; il disegno non deve bloccare il passo fisico di SOFA.

Riunione 2 - fine Fase 2

- Verificare che il cambio modulo E avvenga solo quando cambia il ramo e che resti stabile in pausa.
- Eseguire un ciclo libero completo in modalità SEA; controllare tensione, ΔLs e segno del cavo.
- Verificare che T non sia negativa e che l'avvio non generi precarico imprevisto.
- Controllare che il CSV abbia tutte le colonne valorizzate e che il plotter aggiorni senza rallentare in modo evidente SOFA.
- Provare l'ingresso e l'uscita da contact_hold, anche se l'ostacolo verrà completato nella Fase 3.

9. Fase 3 - Ostacoli, contatto reale e campagna batch

Obiettivo della fase
Aggiungere l'oggetto rigido e quello soffice, verificare la pipeline di collisione e la forza di contatto misurata, quindi automatizzare la campagna di simulazioni senza sostituire SOFA con formule sintetiche.

9.1 Annalisa - generare gli oggetti

1. Creare una sfera di diametro 20 mm (raggio 10 mm) con centro nominale (0, 12,2, 92) mm, come nel piano originale.
2. Esportare la superficie della sfera per il caso rigido in obstacle_rigid.stl.
3. Per il caso soffice esportare una mesh volumetrica tetraedrica obstacle_soft.vtk con qualità verificata.
4. Controllare che la superficie e il volume siano coerenti e che il sistema di coordinate sia lo stesso del tubo.
5. Consegnare ad Anna i due file e uno screenshot con assi, centro e raggio. Non cambiare il centro per 'far toccare' la punta senza documentare la modifica.

9.2 Anna - costruire le collisioni in SOFA

1. Aggiungere una sola collision pipeline. La configurazione di riferimento comprende broad phase, narrow phase, intersezione per prossimità e DefaultContactManager. L'esempio ufficiale usa DefaultPipeline, BruteForceBroadPhase, BVHNarrowPhase, MinProximityIntersection e DefaultContactManager [S6].
2. Configurare il contact manager con response='FrictionContactConstraint' e μ=0,2, dopo avere verificato che il componente sia disponibile nella release installata.
3. Nel nodo Collision del tubo caricare la superficie esterna, creare MechanicalObject e Triangle/Line/PointCollisionModel e mappare la superficie al tubo FEM.
4. Per l'ostacolo rigido caricare STL, creare MechanicalObject e modelli di collisione statici (moving/simulated disattivati secondo API della versione).
5. Per il soffice caricare la mesh volumetrica, creare MechanicalObject, massa, solver e TetrahedronFEMForceField con E=0,005 MPa e ν=0,45.
6. Creare una topologia superficiale triangolare dalla mesh tetraedrica, tramite Tetra2TriangleTopologicalMapping o soluzione equivalente, e aggiungere i modelli di collisione alla superficie mappata [S5].
7. Fissare soltanto una zona di supporto dell'ostacolo soffice: verificare che il BoxROI selezioni un numero di nodi maggiore di zero e che il resto dell'oggetto possa deformarsi.
8. Chiamare realmente dal main tutte le funzioni: setup_collision_pipeline(), build_robot(..., with_collision=True) e add_obstacle(..., obstacle_type=...).
9. Visualizzare i modelli di collisione e verificare che non vi siano superfici mancanti, normali invertite o collisioni tra oggetti che non dovrebbero interagire.

9.3 Anna + Josh - misurare la forza di contatto

1. Abilitare computeConstraintForces nel GenericConstraintSolver.
2. Eseguire una scena semplice con un unico contatto e verificare quale parte di constraintForces corrisponde al contatto. La struttura delle forze va letta e mappata secondo i vincoli creati: non sommare la forza del cavo con quella del contatto.
3. Calcolare la risultante normale di contatto e convertirla in mN: F[mN]=1000 × F[N]. Registrare anche contact_active.
4. Confrontare il segno e l'ordine di grandezza con la reazione attesa in una prova statica controllata.
5. Eliminare completamente la precedente formula artificiale basata su delta_Y > 2,2 mm e k_env.

9.4 Josh - runner batch reale

File: 7_run_campaign.py. Usare la modalità batch ufficiale di runSofa oppure il controllo Sofa.Simulation. I comandi documentati di runSofa comprendono la GUI batch e il numero di iterazioni; SofaPython3 espone init e animate [S1, S2].

1. Creare una funzione run_one_case(params) che scrive la configurazione della prova e avvia il processo SOFA con parametri univoci.
2. Usare runSofa -g batch -n N scena.py (verificando la sintassi con runSofa -h sulla versione installata) oppure uno script Python eseguito nell'ambiente SOFA con Sofa.Simulation.init e animate.
3. Per ogni prova impostare scenario free, rigid o soft; impostare k, c, Fh_molla, Fh_mat; usare il ciclo libero o contact_hold appropriato.
4. Salvare ogni prova in dataset/raw/ con nome univoco. Per esempio free_k_1p2.csv, rigid_k_1p2.csv, soft_k_1p2.csv; nelle prove di sensibilità aggiungere Fh e c al nome.
5. Controllare il codice di uscita, il numero di campioni, durata, colonne, NaN, contatto attivo e parametri effettivamente usati. Una prova fallita deve essere segnata come fallita, non riempita con una curva fittizia.
6. Salvare un file campaign_manifest.csv con run_id, parametri, scenario, data/ora, versione SOFA e stato della prova.

Numero di prove consigliato

Campagna	Combinazioni	Numero configurazioni
Identificazione + validazione	9 valori di k × 3 scenari (free, rigid, soft)	27
Sensibilità	3 k (1,2; 2,4; 4,8) × 3 Fh_molla × 3 c × 3 scenari	81
Totale completo consigliato	Tutte le condizioni della traccia, includendo entrambi gli oggetti	108

I 9 valori di k sono 0,3; 0,6; 0,9; 1,2; 2,4; 3,4; 4,8; 9,6; 50 N/mm. I valori 0,9 e 3,4 sono riservati alla validazione e non devono entrare nel fitting. Le prove di sensibilità usano k=1,2; 2,4; 4,8, Fh_molla=0; 0,45; 0,9 N e c=0,5; 1,5; 5 N·s/mm. Se il tempo di calcolo è un vincolo, il gruppo può concordare una campagna ridotta con il docente, ma deve dichiarare quali combinazioni sono state omesse.

Riunione 3 - fine Fase 3

- Aprire in modalità grafica una scena con oggetto rigido e una con oggetto soffice: verificare contatto visivo e deformazione del soffice.
- Verificare che la forza registrata provenga dai vincoli di contatto e non dal controller SEA.
- Eseguire una prova di mantenimento: forza nel range 10-50 mN, obiettivo 20 mN, contatto continuo per 5 s.
- Lanciare da terminale una prova batch singola e controllare il CSV prima di lanciare tutta la campagna.
- Eseguire 2-3 combinazioni di prova; soltanto dopo l'esito positivo lanciare la matrice completa.

Checklist per passare alla Fase 4
Le scene rigida e soffice funzionano; la forza di contatto è misurata dal solver; il mantenimento è documentato su 5 s; il runner esegue scene SOFA reali; i CSV contengono i parametri corretti e non dati sintetici.

10. Fase 4 - Analisi, modello ridotto, validazione e grafici

Obiettivo della fase
Calcolare H dai CSV reali, identificare il modello ridotto sui sette k di identificazione, validarlo sui due k non usati, stimare la stabilità del contatto e determinare gli intervalli di rigidezza che soddisfano entrambi i target.

10.1 Annapia - calcolo dell'isteresi H

File: 8_signal_analysis.py. Per ogni ciclo in moto libero separare la salita e la discesa e confrontarle agli stessi valori di u.

```
H = max_u |DeltaY_load(u) - DeltaY_unload(u)| / DeltaY_max * 100
```

1. Leggere un CSV reale di moto libero e verificare che siano presenti sia valori di direzione +1 sia -1.
2. Separare i dati di carico e scarico; escludere il punto di inversione duplicato o gestirlo in modo esplicito.
3. Ordinare ciascun ramo per u e interpolare soltanto sul dominio comune di u, senza extrapolare fuori dall'intervallo misurato.
4. Calcolare il massimo scarto verticale tra i due rami e dividerlo per ΔY_max come definito dalla traccia.
5. Calcolare H(50) dalla prova di attuazione diretta e riportare H_rel(k)=100×H(k)/H(50). Il target è H_rel≤50%.
6. Se H(50) è nullo o quasi nullo, segnalare il dato come non valido: non impostare automaticamente il rapporto a zero.

10.2 Annapia - stabilità del contatto

```
Oscillazione_pct = (F_max - F_min) / F_media * 100
Stabile = (Oscillazione_pct < 30) AND (F_min > 0)
```

1. Leggere il CSV di contatto e verificare che contenga almeno gli ultimi 5 secondi di mantenimento.
2. Individuare l'inizio della finestra solo dopo raggiungimento del contatto e assestamento. Non includere la rampa di avvicinamento nei 5 s di stabilità.
3. Calcolare F_media, F_min, F_max e oscillazione picco-picco relativa alla media.
4. Dichiarare stabile soltanto se oscillazione <30%, F_min>0 e contact_active è sempre vero nella finestra.
5. Salvare una riga di risultato per ogni scenario e k in summary_campaign.csv; per la sensibilità salvare sensitivity_campaign.csv.

10.3 Annalisa - identificare il modello ridotto H(k)

Usare il modello proposto nella traccia, che considera la rigidezza equivalente di tubo e SEA in serie:

```
K_eq(k) = (K_t * k) / (K_t + k)
H_model(k) = [2 * (Fh_molla + (k/(K_t+k))*Fh_mat)
            / (K_eq(k) * u_max)] * 100
```

1. Leggere H_free dai CSV riassuntivi soltanto per k=0,3; 0,6; 1,2; 2,4; 4,8; 9,6; 50 N/mm.
2. Escludere k=0,9 e 3,4 dal fitting: sono i due punti di validazione.
3. Usare curve_fit con stime iniziali e limiti fisicamente ragionevoli, dichiarando u_max e unità.
4. Riportare Kt, Fh_molla, Fh_mat, errore/residui sui punti di identificazione e incertezza/covarianza se disponibile.
5. Non inserire un Kt_true preimpostato nel generatore di dati: il Kt deve essere stimato dai dati SOFA.

10.4 Annapia - validazione indipendente

1. Leggere H_SOFA per k=0,9 e k=3,4 dal dataset prodotto dal runner.
2. Calcolare H_pred con i parametri identificati da Annalisa, senza rifare il fitting includendo quei due punti.

3. Calcolare errore_relativo = |H_SOFA-H_pred|/|H_SOFA|×100. Se il denominatore è prossimo a zero, dichiarare l'errore relativo non definito e riportare anche errore assoluto.
4. Il requisito è errore strettamente inferiore al 15% su entrambi i punti. Se non è rispettato, non modificare a mano i dati: discutere il modello, la qualità dei dati e la procedura di fitting.

10.5 Modello ridotto del contatto: integrazione necessaria

Il modello H(k) sopra non contiene una dinamica di contatto. Per poter prevedere anche la stabilità senza nuove simulazioni, il gruppo deve aggiungere una relazione ridotta F_contatto(t) per ciascun ambiente (rigido e soffice), per esempio usando rigidezza e smorzamento equivalenti di contatto identificati sui soli k di identificazione. La forma scelta e i parametri devono essere esplicitati e validati: non basta interpolare con una retta i valori di oscillazione e chiamarla previsione fisica.

Il modello aggiuntivo non deve usare i dati di k=0,9 e 3,4 per essere tarato se quei valori sono dichiarati di validazione. Chiarire col docente se è accettata una parte empirica separata per la stabilità oppure se richiede un unico modello ridotto accoppiato.

10.6 Anna - intervalli ottimali e grafici principali

1. Leggere H(k), H(50) e la previsione/valutazione della stabilità per rigido e soffice.
2. Applicare insieme le condizioni H(k)≤0,5×H(50) e oscillazione contatto<30%, oltre a F_min>0.
3. Calcolare separatamente gli intervalli validi per oggetto rigido, soffice e intervallo comune.
4. Se i punti validi formano più segmenti disgiunti, riportare tutti i segmenti e non soltanto min e max globali.
5. Produrre grafico H(k) con dati di identificazione, curva del modello e punti di validazione; secondo grafico con oscillazione del contatto rigido e soffice e soglia del 30%.
6. Limitare l'intervallo di previsione al dominio supportato dai dati, oppure motivare e segnalare chiaramente ogni estrapolazione.

10.7 Josh - grafici dei cicli, serie temporali e controllo qualità

File da creare: 11_plot_cycles_and_contact.py. Josh è responsabile dei grafici temporali e della verifica della campagna nella fase conclusiva. Non ricalcola H né sostituisce l'analisi di Annapia: usa i CSV reali e i risultati riassuntivi già prodotti.

1. Attendere i CSV grezzi, campaign_manifest.csv, summary_campaign.csv e sensitivity_campaign.csv prodotti nelle fasi precedenti. Verificare che i file appartengano alla stessa versione della campagna.
2. Per i cicli in spazio libero, leggere le prove k=1,2 N/mm e k=50 N/mm; separare il ramo di carico (dir=+1) da quello di scarico (dir=-1) e rappresentare ΔY in funzione di u, mantenendo distinguibili i due rami.
3. Per il contatto, leggere le prove contact_hold dell'oggetto rigido e dell'oggetto soffice, usando lo stesso valore di k scelto come compromesso. Rappresentare F_contatto(t) negli ultimi 5 s di mantenimento e aggiungere una linea di riferimento a 20 mN.
4. Usare esclusivamente la forza normale misurata dai vincoli di contatto del solver SOFA. Non ricostruire la forza con formule artificiali, non sostituire i dati mancanti con sinusoidi e non applicare smoothing che nasconda picchi o perdita di contatto.
5. Verificare la qualità dei CSV e del manifest: presenza delle colonne attese, assenza di NaN, passo temporale coerente, durata sufficiente, parametri k/c/Fh corretti, scenario corretto e stato della prova riuscito. Segnalare i run falliti senza trasformarli in risultati validi.
6. Esportare grafici_cicli_e_contatto.png e un report campaign_quality_report.csv con l'esito dei controlli. Annotare quali run sono stati utilizzati in ciascun grafico.

Dipendenze: Josh riceve da Annapia la conferma dei rami di carico/scarico e della finestra di mantenimento; riceve da Anna la selezione dei valori di k e gli scenari di contatto da confrontare. I CSV e il manifest devono provenire dal runner di Josh della Fase 3.

10.8 Grafici finali e loro proprietario

Grafico	Contenuto	Responsabile
H(k)	Dati SOFA, modello ridotto, k di validazione e soglia 50% di H(50).	Anna + Annalisa
Stabilità contatto	Oscillazione rigido/soffice, soglia 30%, range validi.	Anna
Cicli di isteresi	ΔY(u) per k=1,2 e k=50 N/mm, ramo carico e scarico.	Josh; Annapia verifica i rami e i dati
Forza in mantenimento	F_contatto(t) negli ultimi 5 s per rigido e soffice, riferimento 20 mN.	Josh; Annapia fornisce i risultati di stabilità
Sensibilità Fh_molla	H al variare di 0; 0,45; 0,9 N per k=1,2; 2,4; 4,8.	Annalisa
Sensibilità c	Oscillazione contatto al variare di 0,5; 1,5; 5 N·s/mm.	Annalisa + Anna
Qualità campagna	Completezza, durata, passo e coerenza dei parametri dei CSV e del manifest.	Josh

Riunione 4 - chiusura tecnica

- Controllare che H(50) sia calcolato e non nullo; confrontare tutti i sette punti di identificazione.
- Verificare che i due punti di validazione siano realmente esclusi dal fit e che l'errore sia <15%.
- Controllare formula e finestra temporale della stabilità; nessuna perdita di contatto nei 5 s.
- Verificare che il modello preveda anche la stabilità oppure dichiarare chiaramente la parte empirica e confrontarsi con il docente.
- Controllare che gli intervalli di k siano continui, nel dominio dei dati e soddisfino contemporaneamente i due target.
- Esportare figure in alta risoluzione, archiviare CSV e codice usato per generarli.
- Verificare grafici_cicli_e_contatto.png e campaign_quality_report.csv preparati da Josh; controllare che mostrino solo dati reali e run riusciti.

11. Calendario delle riunioni e cosa fare insieme

Riunione	Quando	Obiettivo comune	Decisione per avanzare
0 - Setup	Prima di Fase 1	Versione SOFA, plugin, Python, cartelle, Git, convenzioni e scena minima.	Tutti eseguono la scena minima.
1 - Fine Fase 1	Dopo mesh e scena base	Caricare la mesh, verificare FEM, 40 mm fissi, tendine e tasti.	Tubo si piega nel verso corretto; nessun errore in console.
2 - Fine Fase 2	Dopo SEA, controller, CSV/plotter	Verificare E load/unload, tensione T, ΔLs, ciclo, plot e CSV.	Ciclo libero e logging funzionano; dati fisici plausibili.
3 - Fine Fase 3	Dopo ostacoli e runner	Contatto rigido/soffice, forza dal solver, mantenimento 5 s, prova batch.	Una prova reale per scenario supera i controlli; poi lanciare campagna.
4 - Fine Fase 4	Dopo analisi e modello	H, fit, validazione, stabilità prevista, range k e grafici.	Tutti i target sono verificati o le criticità sono dichiarate.
Finale	Dopo chiusura codice	PowerPoint, video, controllo dei file e prova orale.	Consegna completa e riproducibile.

Schema fisso per ogni riunione settimanale

1. Ogni responsabile mostra il file aggiornato e spiega che cosa ha cambiato.
2. Il gruppo esegue la prova prevista dalla checklist, senza saltare passaggi.
3. Si salvano log, screenshot, CSV e una breve nota con eventuali errori.
4. Si assegna un responsabile e una scadenza interna per ogni problema emerso (la scadenza è una decisione del gruppo, non una stima tecnica di questa guida).
5. Si aggiorna il README con lo stato: completato, in corso, bloccato, non verificato.

Fase finale - PowerPoint massimo 15 slide

Membro	Slide suggerite	Contenuti
Annalisa	1-3	Obiettivo, geometria del tubo/ostacoli, Nitinol e modello ridotto H(k), parametri identificati.
Anna	4-7	Architettura SOFA, mesh FEM, tendine/mapping, collisioni, contatto reale e intervalli ottimali.
Annapia	8-11	Molla SEA, controller, definizione di H, stabilità 5 s e tabella di validazione.
Josh	12-15	Architettura software, LivePlotter, runner batch SOFA, qualità dei CSV, cicli di isteresi e serie temporali della forza di contatto.

Se il limite è 15 diapositive, la slide 15 deve contenere conclusioni e limiti principali, non solo un grafico aggiuntivo. Le slide 1-14 possono essere distribuite in modo flessibile, ma ogni risultato deve mostrare scenario, k, c, isteresi e versione della simulazione quando rilevanti.

Video dimostrativo - circa 2 minuti

1. Mostrare la scena iniziale e indicare i 40 mm fissati e i 60 mm flessibili.
2. Mostrare il carico/scarico manuale e il ciclo automatico con stampa di u, T, ΔLs e posizione punta.
3. Mostrare una prova con oggetto rigido e una con oggetto soffice; evidenziare forza contatto misurata e mantenimento 5 s.
4. Mostrare rapidamente LivePlotter, CSV, curva H(k), intervallo ottimo e verifica di validazione.

12. Matrice finale di conformità alla traccia

Questa matrice serve come checklist prima della consegna. 'Superato' va selezionato soltanto dopo una prova effettiva e archiviata.

Richiesta della traccia	Come la copriamo	Evidenza da conservare
Geometria 40+60 mm e dimensioni tubo	1_generate_meshes.py + controllo mesh	STL/VTK, screenshot, misure.
Mesh superficie e mesh volume	Mesh Gmsh per visuale/collisione e FEM	File e verifica elementi tetraedrici/superficie.
FEM con E carico/scarico e ν=0,33	Anna integra FEM; Annalisa definisce i rami E	Log dei valori E nei due rami.
BarycentricMapping visuale e collisione	Anna costruisce mapping espliciti	Immagine del tubo deformato e superficie allineata.
CableConstraint in forza	Anna configura; Annapia controlla forza e segno	Valore T e risposta della punta.
SEA con k, c e isteresi	Annapia implementa formula e inizializzazione	CSV con u, ΔLs, T e prova di segno.
Controller tastiera e modalità autonoma	Annapia	Video/manuale, free_cycle e contact_hold.
Stampe u, T, ΔLs, posizione punta	Annapia	Log console e unità.
Grafici real-time e CSV	Josh	Screenshot plotter, CSV completo.
Oggetto rigido e soffice; μ=0,2	Annalisa genera; Anna integra	Scene e screenshot delle collisioni.
Forza contatto 10-50mN / target 20mN	Anna integra la misura dal solver; Josh verifica e rappresenta le serie temporali	CSV e serie temporale verificabile.
Hysteresis H in spazio libero	Annapia	Script e summary_campaign.csv.
Stabilità per 5s, oscillazione <30%	Annapia	Finestra di 5s, Fmin>0, contact_active.
Campagna k ident./validazione/sensibilità	Josh	Manifest e CSV SOFA reali.
Modello ridotto Kt, Fh_molla, Fh_mat	Annalisa	Fit, parametri, residui e formula.
Validazione k=0,9 e 3,4; errore <15%	Annapia	Tabella errori calcolati su dati esclusi dal fit.
Previsione stabilità senza nuove simulazioni	Annalisa + Anna	Sottomodello contatto, parametri identificati e validazione.
Intervalli rigido, soffice e comune	Anna	Script, tabella intervalli, grafici e criterio applicato.
PPT max 15 slide + video	Tutti	PPTX, video 2 min e cartella consegna.

Criterio finale di accettazione

Non dichiarare un requisito soddisfatto se manca l'evidenza
Ogni riga della matrice deve avere almeno un artefatto verificabile (file, CSV, log, grafico o video). Se una parte non è stata realizzata, scrivere 'non completata' e non sostituire il risultato mancante con una curva teorica presentata come simulazione.

13. Riferimenti e note di versione

La struttura del progetto e i valori numerici di base derivano dalla traccia e dal documento di organizzazione del lavoro fornito dal gruppo. La notazione è allineata al documento 'Glossario Completo delle Variabili, Parametri e Simboli' caricato dal gruppo, dando priorità ai valori della traccia specifica quando i benchmark generali differiscono. Le indicazioni tecniche su SOFA/SofaPython3 sono state confrontate con la documentazione ufficiale disponibile al 5 novembre 2026.

Documento di partenza

'Organizzazione del lavoro - Progetto SOFA (Robotica Medica)', PDF di 30 pagine caricato dal gruppo. La presente guida ne conserva la struttura in fasi e i parametri, ma corregge la suddivisione e le procedure non conformi.

Documentazione ufficiale consultata

[S1] SOFA - runSofa, avvio scene Python, plugin e modalità batch. Apri documentazione

[S2] SofaPython3 - First Steps e Sofa.Simulation (init/animate). Apri documentazione

[S3] Sofa.Component - GenericConstraintSolver, output constraintForces e computeConstraintForces. Apri documentazione

[S4] SOFA SoftRobots - CableConstraint, valueType, displacement e force. Apri documentazione

[S5] SOFA - Tetra2TriangleTopologicalMapping, conversione superficie da tetraedri. Apri documentazione

[S6] SOFA - DefaultPipeline e componenti della collision pipeline. Apri documentazione

[S7] SofaPython3 - Sofa.Core.Controller e callback degli eventi. Apri documentazione

[S8] SOFA - Mapping e rappresentazioni fisiche, visuali e di collisione. Apri documentazione

[S9] SOFA - TetrahedronFEMForceField. Apri documentazione

[S10] SOFA - MeshVTKLoader. Apri documentazione

Avvertenze finali

- SOFA è un framework modulare: il nome di un componente, il plugin che lo contiene e i Data disponibili possono cambiare tra release. Verificare la versione installata e non copiare alla cieca blocchi di codice presi da documentazioni di release diverse.
- Il piano non sostituisce il confronto con la docente sui punti lasciati indeterminati dalla traccia: densità/massa, convenzione del precarico e del segno della tensione, e natura della previsione di stabilità del modello ridotto.
- Le simulazioni e i dati devono essere riproducibili: conservare configurazione, versione, log, CSV grezzi e script che hanno prodotto i grafici.

Fine guida unica - versione aggiornata del 5 novembre 2026